Autocomplete System Design – Summary

Goal: Build a universal autocomplete system delivering suggestions in <150ms end-to-end, with strong relevance,
reliability, and privacy compliance.

Key Features:
- Client: TypeScript SDK, ARIA-compliant combobox, async & incremental suggestions, trending/recent queries.
- Server: Prefix + fuzzy + semantic recall, personalized ranking, filtering, synonyms, analytics, admin APIs.
- Non-functional: 99.95% availability, p95 backend <80ms, six-nines durability, GDPR/CCPA compliance.

Architecture:
- Client SDK, Edge caching & validation, Query Service, Index Service, Ingestion, Personalization, Analytics, Admin, Metadata Store, Monitoring.
- Storage: In-memory indices, Redis, RocksDB/OpenSearch, object storage.

Algorithms:
- DFA/trie prefix match, fuzzy tolerance, semantic embeddings.
- Ranking: exactness, recency, popularity, personalization.
- Typos: normalization + keyboard distance.
- Personalization with strict opt-in/residency rules.

Scaling & Capacity:
- Shard by tenant/prefix space, adaptive caching for hot prefixes.
- 1M QPS globally; 90% from edge caches.
- Updates: 50K/sec, bursts up to 200K.
- Cost per region: tens to hundreds of thousands monthly.

APIs:
- Query API: GET /v1/query
- Admin APIs: manage indices, synonyms, rules.
- Ingestion APIs: batch upserts, deletes, streaming.
- Analytics APIs: query/selection events, reports.

Ops & Testing:
- Observability: OpenTelemetry, Prometheus, Grafana.
- CI/CD: Git pipelines, Terraform, canary/blue-green deploys.
- Testing: unit, property, load, replay, failure drills.
- Risks: latency vs flexibility, cache staleness, personalization vs privacy.

Roadmap:
- MVP (4 weeks): Prefix search, edge cache, batch ingestion.
- Phase 2 (2 months): Fuzzy, synonyms, rules, personalization, analytics.
- Phase 3 (3–6 months): Multi-region active-active, semantic recall, advanced ranking, enterprise features.